

4] Compare "original" & "reversed"

▲ Important Points → Negative Number are not palindromes

• $\%10$ gets the last digit.

• $/10$ removes the last digit

• "long long" helps avoid overflow while reversing.

▲ Complexity → Brute Force → $O(n)$ & $O(n)$

Main Solution → $O(\log_{10} x)$ & $O(1)$

• Easy → #13 → Roman To Integer

▲ Problem Statement → Given a Roman numeral string, convert it into an integer.

▲ Main Solution → Use an "unordered_map" to store Roman symbols & their values. Compare each symbol with the next one.

• Current < Next → Subtract

• Otherwise → Add

• Add the last symbol at the end.

▲ Brute Force → Check for special cases like "IV", "IX", "XL", etc., separately & calculate the result accordingly.

▲ Algorithm → 1] Create an "unordered_map" with Roman symbols & values.

2. Loop from "0" to "S.length() - 2"

3. Compare the current value with the next value

4. If $\text{current} < \text{next}$, subtract it; otherwise, add it.

5. Add the last symbol's value

6. Return the result.

▲ Important Points: • Use "unordered_map" for "Roman Symbol - Value"

• If a smaller value comes before a larger value, subtract it.

• Otherwise, add it.

• The last symbol is always added separately.

▲ Complexity: Brute Force: $O(n)$ & $O(1)$

Main Solution: $O(n)$ & $O(1)$